UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Class design

Readings: ( :chapt. 3, 8)

1

Check point: what's the output, if any?

```
class Point {
   private int x, y;
   public void x(int x)      {this.x = x;}
   public Point() {x=0; y=0;}
   public String toString() {return x+"@"+y;}
}

public class Check{
   void main() {
        Point origin = new Point();
        IO.println("Origin: "+origin);
        Point p = origin;
        p.x(1);
        p=null;
        IO.println("Origin: "+origin);
        IO.println(p); }}
```


2

References

- In Java a variable contains either a value of primitive data type or a (object) reference
- If the variable is of a primitive data type, assigning means copying the value resulting from the right expression to the contents of the variable on the left. In the example,

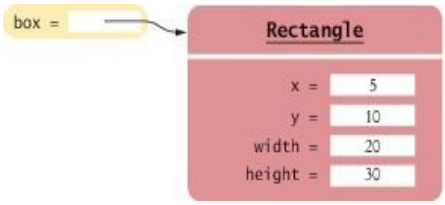
```
int i, j =10;
i=j;
```

both i and j have independent storage spaces, that after the assignment will have the same value

Object references

- If the type of the variable is a class then
  - It does not hold the actual object;
  - It holds the memory location of an object, i.e., the object reference

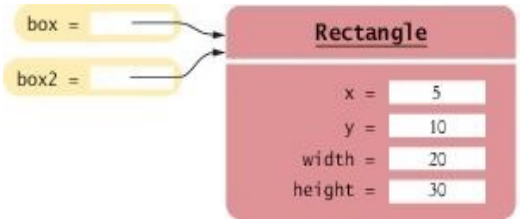
Rectangle box = new Rectangle(5, 10, 20, 30);



Object references

```
Rectangle box = new Rectangle(5, 10, 20, 30);
Rectangle box2 = box;
```

After assignment both variables box and box2 refer to the same object of the class Rectangle



Argument passing

- In Java argument passing is allways done by value.
- The values of primitive data types are copied to the function formal parameters; for other types, what are copied are the references (not the objects themselves). This is equivalent to reference argument passing.

First goals



- Learn how to choose appropriate classes for a given problem
- Learn how to find a data representation for a class
- Learn the naming convention for classes
- Understand the concept of class invariant
- Understand the concept of cohesion
- Learn to minimize dependencies and side effects

7

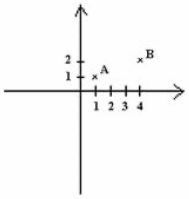
On classes

- A class represents a *single* concept useful for the problem
- A class has an API – (application program) interface, *i.e.,* the set of all its public members (methods or variables)
- The class API is *cohesive* if all of its members are related to the concept that the class represents
- Members of a cohesive team have a common goal

8

How to name a class?

- The name of a class should be a *noun* that describes the concept, such as
  - Point
  - Rectangle
  - BankAccount
  - Person
  - Scanner



- By convention, the name is *singular* and begins with a capital letter

9

Mutable and immutable classes

- A *mutator* is a method that changes the state of an object. Ex: `setX(int x)`
- An *accessor* is a method that asks an object to compute a result, without changing the state. Ex: `getX()`
- An *immutable* class has no mutator methods. Ex: No method in the `String` class can modify the contents of a string.
- References to objects of an immutable class can be safely shared

10

The reference *this* and the receiver

- In the code

```
Point p = new Point(0,1);
p.setX(1);
```

the object referred by `p` – the so-called *receiver* – is passed implicitly to the function `setX(int x)` and can be assessed using the keyword *this*

```
class Point {
    private int x, y;
    public void setX(int x) {
          this.x = x;
    }
     // …
```

11

The receiver

```
class Point {
   static int Dimension = 2;
   private int x, y;
   public getx() { return x);
}
```

- Can be an object:

receiver

```
Point p = new Point(0,1);
p.setX(1);
```

- Or a class:

```
dim = Point.Dimension;
```

12

The receiver and the message

```
Point p = new Point(0,1);
p.setX(1);
```

message

```
dim = Point.Dimension;
```

13

Static (or class) variables

- A static variable hold a unique value (or reference) for all objects of the class where it is defined

```
class Point {
   static int Dimension = 2;
   private int x, y;
   public getx() { return x);
}
```

```
dim = Point.Dimension;
```

14

Static (or class) variables

- A static variable hold a unique value (or reference) for all objects of the class where it is defined

```
class A {
   private static int id=0;
   private int data;
   public A(int d) {
      id++;
      data = d;
   }
   int getData() { return data; }
   static int getID() { return id;}
}
```

```
A a = new A(25);
A b = new A(434);

int n  = b.getData(); //434
int id = b.getID();   //2
```

15

Static (class) methods

- Methods can also be declared as static
- Static methods do not need an object to be called

```
class A {
   private static int id=0;
   private int data;
   public A(int d) {
      id++;
      data = d;
   }
   int getData() { return data; }
   static int getID() { return id;}
}
```

```
A a = new A(25);
A b = new A(434);

int n  = b.getData(); //434
int id = b.getID();   //2
int jd = A.getID();   //2
```

16

More on overloading

- In Java :
  - Variable names do **not** clash with method names
  - we can give to the setters and getters the same name of variables

```
class Point {
    private float x, y;

    public void x(float x) { this.x = x; }
    public float x() { return this.x; }


    // …
}
```

17

More on overloading

- since getters and setters have different signatures they can have the same name

```
class Point {
    private float x, y;

    public void x(float x) { this.x = x; }
    public float x() { return this.x; }


    // …
}
```

```
Point p = new Point(0.0f,1.0f);

p.x(1.0f);
float x0 = p.x();
```

18

Side effects

- A side effect of a method is any externally observable data modification produced by the method
- Setters have a side effect, namely the modification of the implicit variable as in

```
Point p = new Point(0,1);
p.setX(1);
```

- In general, a method should not modify its parameters
- Minimize side effects in methods

19

UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Class design

Computing with rationals: the class Fraction

Readings: ( :chapt. 3, 8)

20

Computing with rationals

- Develop a program able to perform basic arithmetic operations and comparisons with rational numbers.

  Examples of the required operations are:

  i) 1/4 + 1/4 -> 1/2
  ii) -(1/2) * 1/2 -> - 1/4
  iii) 1/2 > 1/3 -> true
  iv) 2/4 = 1/2 -> true

21

Development of the class Fraction representing rationals

- Representation:
  n – numerator
  d – denominator
  d>0
  gcd (|n|, d) = 1 (máximo divisor comum – gcd in English) Ex. 2/4 == 1/2

  Class invariant
  (condição invariante de instância)

- Under the above invariant, operations can be given by:
  n1/d1 + n2/d2 = (n1d2 + n2d1) / (d1 * d2);
  n1/d1 – n2/d2 = (n1d2 - n2d1) / (d1 * d2);
  n1/d1 * n2/d2 = n1 * n2 / (d1 * d2);
  (n1/d1) / (n2/d2) = n1* d2 / (n2* d1); with n2 ≠ 0
  (n1/d1) == (n2/d2) <=> n1 == n2 and d1 == d2
  n1/d1 < n2/d2 <=> n1d2 < n2d1

22

Class invariant

- A class invariant for a class *C* is the condition (possibly a set of assertions) that every instance of *C* must satisfy at all times.

  That is, immediately after its creation and before and after a public method call (by a client)

23

Assertions

- Formally, assertions are first-order logic formulae
- In a programming language, assertions are computable Boolean expressions that can contain variables, arithmetic and boolean statements and, possibly, user-defined functions

24

Class invariants

- Where to enforce class invariants?
  - ***right when the object is created: in the constructor***
- Additionally in this class no mutator should be defined

  public int setNumerator (int n) { num = n; }
- In general, if mutators are really needed they must also enforce class invariants.

25

A possible client

```
class Main {
        void main() {
                Fraction f = new Fraction(4, 6);
                Fraction g = new Fraction(1, -2);
                Fraction s = new Fraction();
                s = f.mux(g);
                boolean b = f.less(g);
                IO.println(s);
                IO.println(b);
        }
}
```

No native support in Java for operator (e.g., *, <) overloading; a compiler plugin is required.

26

```
/** @version 1.0 */
public class Fraction {
   private int num, den;
   public Fraction() { this(0); }
   public Fraction(int n) { num= n; den = 1; }
   public Fraction (int n, int d) {
      if (d == 0) {
          System.err.print("Fraction.denominator() cannot be zero");
          System.exit(0);      // We will have to improve this later
      }
      if (d < 0) {
         this.num = -n;
         this.den = -d;
      }  else {
       num = n;          // same as this.num =n;
       den = d;
      }
      int k =gcd(Math.abs(num), den);
      num /= k;
      den /= k;
   }
```

Class invariant

(condição invariante de instância)

d > 0

gcd (|n| , d) = 1

eg.: 8/16 == 1/2, k=8

27

```
/** @version 1.0 */
public class Fraction {
   private int num, den;
   public Fraction() { this(0); }
   public Fraction(int n) { num= n; den = 1; }
   public Fraction (int n, int d) {
      if (d ==0) {
          System.err.print("Fraction.denominator() cannot be zero");
          System.exit(1);       // We will have to improve this later
      }
      if (d<0) {
         this.num = -n;
         this.den = -d;
      }  else {
       num = n;          // same as this.num =n;
       den = d;
      }
      int k =gcd(Math.abs(num), den);
      num /= k;
      den /= k;
   }
```

Referencing the receiver with *this*

28